

P3660R0

Improve `reference_wrapper` Ergonomics

Published Proposal, 2025-03-12

This version:

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3660r0.html>

Author:

[Jeremy Rifkin](#)

Audience:

SG18

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

<https://github.com/jeremy-rifkin/proposals/blob/main/cpp/reference-wrapper-improvements.bs>

Abstract

`reference_wrapper` is an existing practice to fill a gap between nullability and reassignability while also providing a well-behaved reference object. This paper proposes a small set of ergonomic improvements.

Table of Contents

- 1 Introduction
- 2 Motivation
- 3 Alternatives
- 4 Proposal
- 5 Proposed Wording

References

Normative References

Informative References

§ 1. Introduction

References in C++ are aliases to objects. They are not guaranteed to occupy storage, cannot be reassigned, and are not objects themselves. Pointers, on the other hand, point to objects while being reassignable objects. While references and pointers serve much the same purpose, references provide an important guarantee that they will never refer to a null object. Thus, there is a gap in core language and library support for representing non-nullable but reassignable indirection:

	Not reassignable	Reassignable
Nullable	<code>T* const</code>	<code>T*</code>
Non-nullable	<code>T&</code>	-

When faced with a problem where such a non-nullable reassignable reference type would be useful, developers must either switch to pointers or use another wrapper type with these properties. Due to its availability, `std::reference_wrapper` has emerged as an existing practice for this use case. However, because it was designed for certain metaprogramming uses requiring references that behave like regular objects as opposed to serving as a more general utility type, it is not very ergonomic. The two most notable shortcomings are that the name is long and cumbersome and the only ways to access the referenced object are a conversion operator or `.get()` instead of a forwarding member access operator.

Ergonomic limitations of `reference_wrapper` have led some developers to create their own reference wrapper-like types that are more ergonomic in lieu of `reference_wrapper`'s shortcomings. For example, Barry Revzin shared on the [std-proposals](#) mailing list in [September 2024](#):

In reply to a question asking if `.get()` was not already sufficient:

For what it's worth, we have a template that inherits from `reference_wrapper<T>` specifically for this reason. After a lot of use, `r.get().f()` is pretty verbose compared to `r->f()`.

Well that and the name `reference_wrapper<T>` is also very verbose so ours is just `Ref<T>`.

Both the name and constantly having to call `.get()` results in this utility being quite cumbersome to use. `std::optional` would be similarly cumbersome if developers had to constantly call `.value()` as opposed to being able to use `operator->` for member access.

This paper proposes a small set of ergonomic improvements to address these limitations and transform the type from a metaprogramming utility to a more broadly applicable vocabulary type that closes the gap between nullability and reassignability.

§ 2. Motivation

References are often preferred in C++ because they do not require careful thought about nullability. However, there are times when references can't be used due to not behaving like objects and not being reassignable. For example, it's generally not possible to implement a copy or move assignment operator for a class with non-static reference members and implicitly-declared copy and move assignment operators are defined as deleted. Additionally, due to not behaving like normal objects, all of the following are ill-formed: `int& arr[2]`, `std::array<int&, 2>`, `std::vector<int&>`, `std::variant<int&, float&>`, and `std::initializer_list<int&>`. Notably, `std::optional<T&>` is also ill-formed, however, [P2988] is on track for acceptance. Whenever any of the aforementioned constructs are desired, or a programmer would simply like to reassign a `T& ref`, the limitations of references result in either needing to switch to a pointer or another wrapper type.

Pointers work as replacements for references when object-like behavior or reassignability is desirable. However, they do not reflect non-nullability in the type system. Losing this valuable information in the type system has implications with bug-proneness and may also lead to unnecessary null checks out of an abundance of caution in code paths where non-nullability cannot be readily shown. This can increase complexity and clutter code.

The desire for a non-nullable, reassignable, well-behaved reference has led many to reach for `reference_wrapper` as a standard utility with these semantics. For example, a highly upvoted comment on [Why does std::optional not have a specialization for reference types?](#) mentions using `reference_wrapper` as a workaround.

While `reference_wrapper` was initially added for certain metaprogramming uses, it has become a fairly widespread pattern that can be seen in codebases ranging from hobby projects to large and established codebases such as Chromium. A cursory GitHub code search for `reference_wrapper` in open-source C++ code at the time of writing yields 45k results: [Code Search](#). Many of these uses are:

- `std::reference_wrapper<T>` as a non-static data member

- `std::vector<std::reference_wrapper<T>>`
- `std::optional<std::reference_wrapper<T>>`
- `std::map<K, std::reference_wrapper<V>>`
- Use in `std::variant`
- Use in other containers

The first few pages of results suggest these categories account for the vast majority of use of `reference_wrapper` in open-source C++ code.

Due to its semantics and status as an existing practice for this use case, it's fitting to expand the intended use case of `reference_wrapper` and improve its ergonomics as a general utility instead of introducing a new vocabulary type for this use case.

§ 3. Alternatives

Alternatives to `reference_wrapper` for this use case include:

1. Tell developers to just use pointers
2. Add a `std::not_null` pointer wrapper to fill the gap instead

	Not reassignable	Reassignable
Nullable	<code>T* const</code>	<code>T*</code>
Non-nullable	<code>T&</code>	<code>reference_wrapper</code> , <code>not_null</code>

3. Add first-order language support for reassignable references or references that behave as normal objects

Pointers are undesirable for reasons mentioned earlier.

There is prior art for `not_null` in the form of `gsl::not_null`. This wrapper type fundamentally mimics a pointer, though, and has constructors taking pointers that are null-checked at runtime. `reference_wrapper`, on the other hand, is designed to model a reference: It's constructed only by a reference, and thus, non-nullability is ensured as an invariant at the type system level.

First-order language support for references that behave like objects and are assignable is a possibility, but, it would be a massive change to the language. Instead, improving `reference_wrapper` for this use case seems preferable.

§ 4. Proposal

This paper proposes three changes:

1. Add `template<typename T> using reference = reference_wrapper<T>;` as an alias
2. Add an `operator->` overload for forwarding member access as well as an `operator*` overload for symmetry
3. Provide `reference_wrapper` and the `reference` alias through the `<utility>` header

`std::reference` is chosen as a shorter alias for `reference_wrapper` due to `std::ref` already being used.

Expanding the forwarding operations for `reference_wrapper` follows the precedent of [P2944], which added forwarding comparison operators to `reference_wrapper`. While `operator->` is typically associated with pointers and pointer-like types (such as iterators or smart pointers), it is the only option for member access forwarding in lieu of `operator..`. This use of `operator->` isn't without precedent, and it isn't uncommon even for types that aren't pointer-like. For example, all of the following have `operator->` overloads for convenience: `std::optional`, `std::expected`, or types like `proxy_holder/arrow_proxy` (stackoverflow) or `pilfered` (boost json, [P0308]). `std::optional` is arguably similar to a pointer in that it is nullable; however, `std::expected` and the proxy/wrapper types are less so. Fundamentally, the main commonality between all these types isn't nullability but rather serving as proxies to other objects via either wrapping or indirection. As such, `operator->` is fitting for `reference_wrapper`.

With the expanded use case for `reference_wrapper`, it is better provided through `<utility>` so that `<functional>` does not have to be pulled in just to use the type. This follows the precedent of [P0472], which added `monostate` to `<utility>`. This will require that implementations either `#include <utility>` from `<functional>` or put `reference_wrapper` in a common implementation header that can be included in both places.

§ 5. Proposed Wording

Proposed wording relative to [N4950]:

Move [refwrap] and subsections to [utility.refwrap].

Move from [functional.syn] to [utility.syn]:

```
// [utility.refwrap], reference_wrapper
template<class T> class reference_wrapper;

template<class T> constexpr reference_wrapper<T> ref(T&) noexcept;
template<class T> constexpr reference_wrapper<const T> cref(const T&) noexcept;
template<class T> void ref(const T&&) = delete;
template<class T> void cref(const T&&) = delete;

template<class T>
    constexpr reference_wrapper<T> ref(reference_wrapper<T>) noexcept;
template<class T>
    constexpr reference_wrapper<const T> cref(reference_wrapper<T>) noexcept;

// [utility.refwrap.common.ref], common_reference related specializations
template<class R, class T, template<class> class RQual, template<class> class TQual>
    requires see below
struct basic_common_reference<R, T, RQual, TQual>;

template<class T, class R, template<class> class TQual, template<class> class RQual>
    requires see below
struct basic_common_reference<T, R, TQual, RQual>;
```

Add to [utility.syn]:

```
// [utility.refwrap], reference_wrapper
template<class T> class reference_wrapper;

template<class T> constexpr reference_wrapper<T> ref(T&) noexcept;
template<class T> constexpr reference_wrapper<const T> cref(const T&) noexcept;
template<class T> void ref(const T&&) = delete;
template<class T> void cref(const T&&) = delete;

template<class T>
    constexpr reference_wrapper<T> ref(reference_wrapper<T>) noexcept;
template<class T>
    constexpr reference_wrapper<const T> cref(reference_wrapper<T>) noexcept;

template<class T> using reference = reference_wrapper<T>;
```

```
// [utility.refwrap.common.ref], common_reference related specializations
template<class R, class T, template<class> class RQual, template<class> class TQual>
    requires see below
struct basic_common_reference<R, T, RQual, TQual>;

template<class T, class R, template<class> class TQual, template<class> class RQual>
    requires see below
struct basic_common_reference<T, R, TQual, RQual>;
```

Add a paragraph at the end of [functional.syn]:

The class templates, function templates, and alias template defined in [utility.refwrap] are available when `<functional>` is included.

Add to [utility.refwrap.general]:

```
namespace std {
    template<class T> class reference_wrapper {
    public:
        // types
        using type = T;

        // [utility.refwrap.const], constructors
        template<class U>
            constexpr reference_wrapper(U&&) noexcept(see below);
        constexpr reference_wrapper(const reference_wrapper& x) noexcept;

        // [utility.refwrap.assign], assignment
        constexpr reference_wrapper& operator=(const reference_wrapper& x) noexcept;

        // [utility.refwrap.access], access
        constexpr operator T& () const noexcept;
        constexpr T& get() const noexcept;
        constexpr T* operator->() const noexcept;
        constexpr T& operator*() const noexcept;

        // [utility.refwrap.invoke], invocation
        template<class... ArgTypes>
            constexpr invoke_result_t<T&, ArgTypes...> operator()(ArgTypes&&...)
                noexcept(is_nothrow_invocable_v<T&, ArgTypes...>);
```

```

// [utility.refwrap.comparisons], comparisons
friend constexpr bool operator==(reference_wrapper, reference_wrapper);
friend constexpr bool operator==(reference_wrapper, const T&);
friend constexpr bool operator==(reference_wrapper, reference_wrapper<T>);

friend constexpr auto operator<==(reference_wrapper, reference_wrapper);
friend constexpr auto operator<==(reference_wrapper, const T&);
friend constexpr auto operator<==(reference_wrapper, reference_wrapper<T>);
};

template<class T>
    reference_wrapper(T&) -> reference_wrapper<T>;
}

```

Add to [utility.refwrap.access]:

```
constexpr T* operator->() const noexcept;
```

Returns: `addressof(get())`.

```
constexpr T& operator*() const noexcept;
```

Returns: The stored reference.

§ References

§ Normative References

[N4950]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 10 May 2023. URL: <https://wg21.link/n4950>

§ Informative References

[P0308]

Valueless Variants Considered Harmful. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0308r0.html>

[P0472]

Put std::monostate in <utility>. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0472r3.pdf>

[P2944]

Comparisons for reference_wrapper. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2944r3.html>

[P2988]

std::optional<T>. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2988r8.pdf>